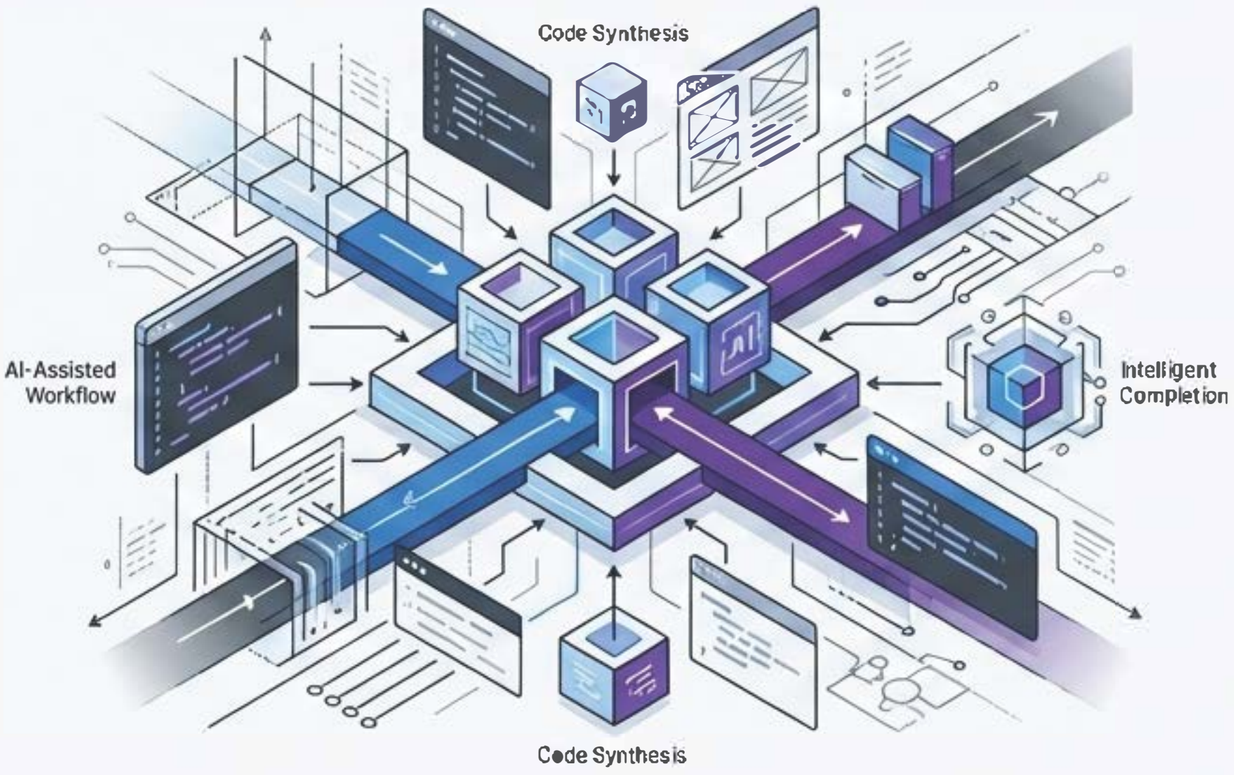
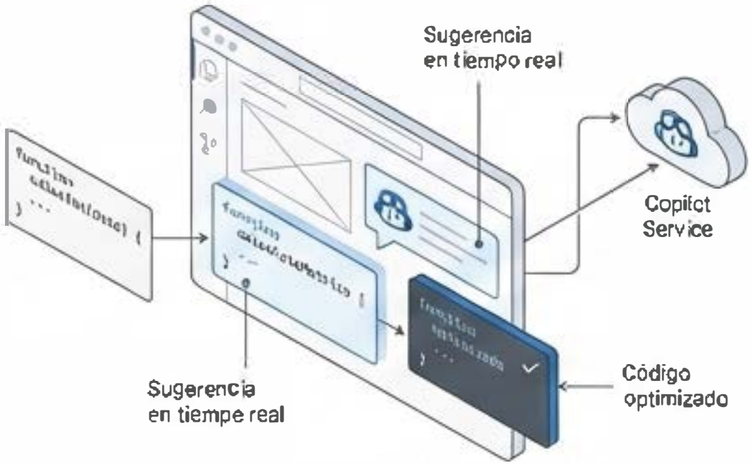
GitHub Copilot en VS Code

Conceptos Centrales y Mejores Prácticas para el Desarrollo con IA



FLUJO DE TRABAJO INTEGRADO



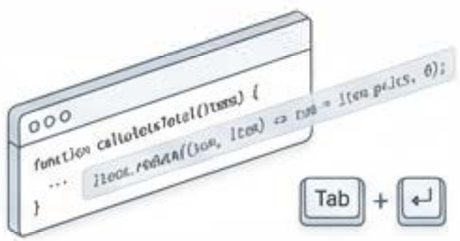
MEJORES PRÁCTICAS

- **Contexto Claro:** Proporcionar comentarios y nombres descriptivos. `// Función para procesar datos de usuarios`
- **Iteración Rápida:** Evaluar y refinar sugerencias continuamente.
- **Validación de Código:** Revisar siempre el código generado por IA.
- **Seguridad Primero:** Evitar secretos o datos sensibles en los prompts.
- **Comandos Específicos:** Utilizar `/explain`, `/fix`, `/tests` para tareas dirigidas.

El Espectro de Asistencia de IA

Ascendiendo hacia la autonomía: desde sugerencias contextuales hasta agentes de ejecución autónoma.

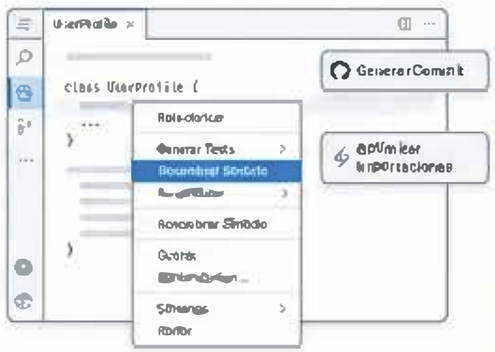
1. Sugerencias en Línea



Autocompletado continuo (Ghost text). Ideal para mantenerse en el flujo de trabajo.

Ideal para: Flujo continuo, reducción de tipeo, correcciones rápidas.

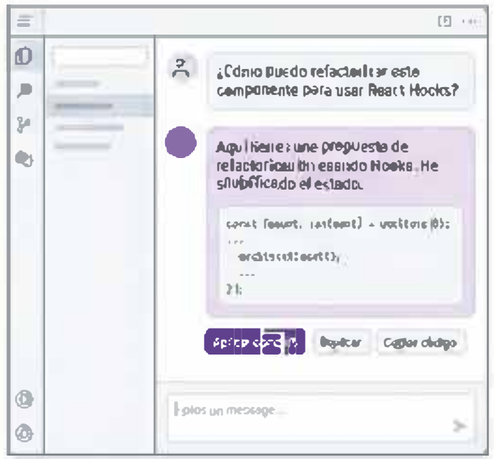
2. Acciones Inteligentes



Tareas de un solo clic integradas en la interfaz (ej. generar commits, renombrar símbolos).

Ideal para: Tareas repetitivas, gestión de código, acciones rápidas sin cambiar de contexto.

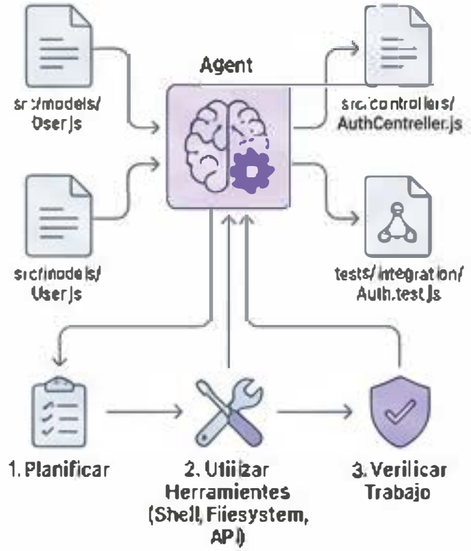
3. Chat



Interfaz conversacional. Preguntas, ideas y refactorización guiada.

Ideal para: Exploración, aprendizaje, resolución de problemas complejos, lluvia de ideas.

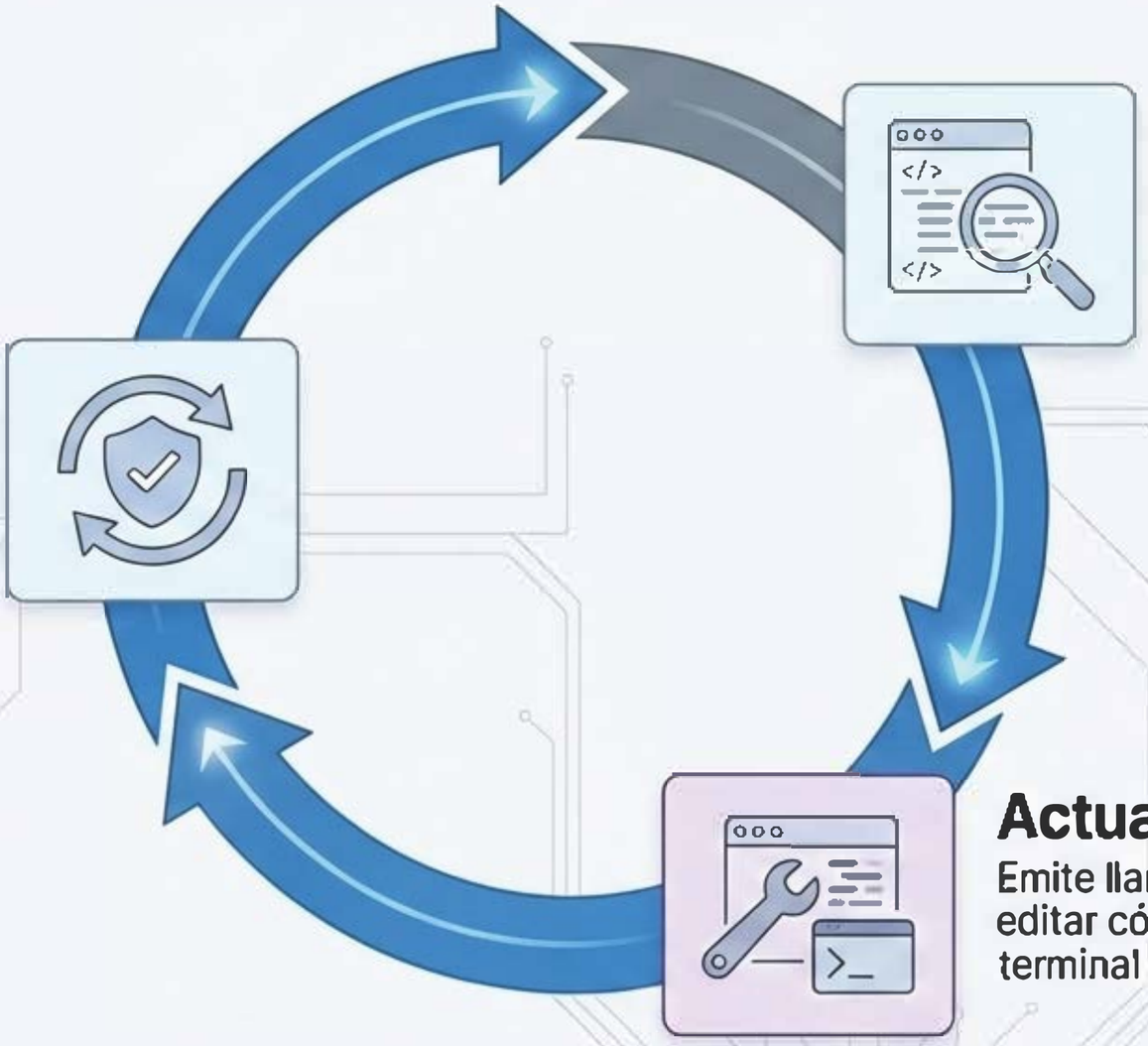
4. Agentes



Ejecución autónoma multi-archivo. Planifican, utilizan herramientas y verifican su propio trabajo.

Ideal para: Proyectos completos, migración de código, desarrollo de características complejas de principio a fin.

Anatomía del Agente: Bucle de Acción



Entender

Lee archivos, busca en la base de código y consulta documentación para contextualizar el problema.

Actuar

Emite llamadas a herramientas para editar código, ejecutar comandos de terminal o instalar dependencias.

Validar

Ejecuta pruebas y revisa errores. Si hay fallos, se auto-corrige y repite el bucle.

Consejo Pro:

Los agentes no solo predicen texto; interactúan activamente con tu entorno de VS Code a través de herramientas integradas.

Entendiendo el Contexto: Cómo se Construye el Prompt

Tipos de Agentes y Selección de Modelos

Entorno del Agente

Local / Interactivo: Ejecución en VS Code. Feedback inmediato.

Background: Tareas asíncronas para no bloquear la interfaz.

Cloud: Infraestructura remota de GitHub. Ideal para colaboración.

Third-Party: Integración nativa de Claude Code y OpenAI Codex.

Selección de Modelos

Rápidos (Fast): Autocompletado, refactorizaciones simples, boilerplate.

Razonamiento (Reasoning): Arquitectura, debugging complejo, planificación.

BYOK (Bring Your Own Key): Uso de claves de API propias para modelos locales o experimentales sin límites estándar.

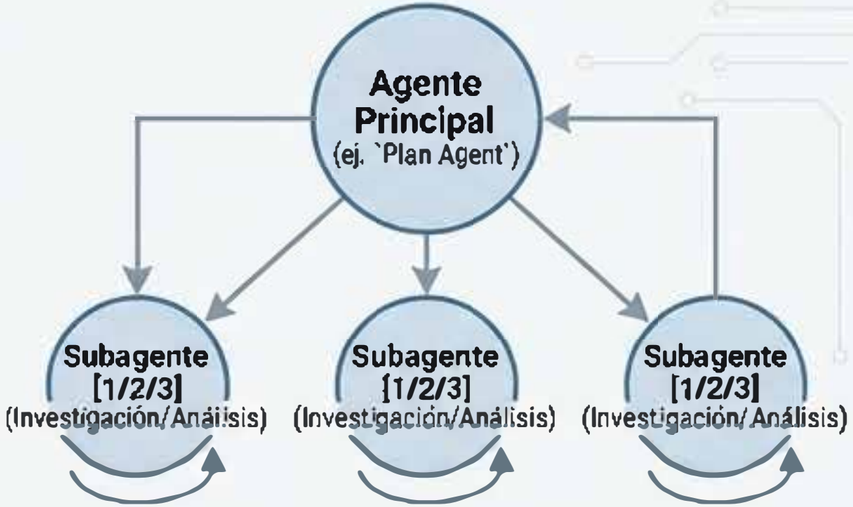
Escalabilidad: Memoria y Subagentes

La Memoria del Agente

Memory Tool: Notas locales almacenadas en tu máquina (`alcance de usuario`, `repositorio` o `sesión`).

Copilot Memory: Alojado en GitHub, comparte conocimientos del repositorio entre el editor, `CLI` y revisiones de PRs.

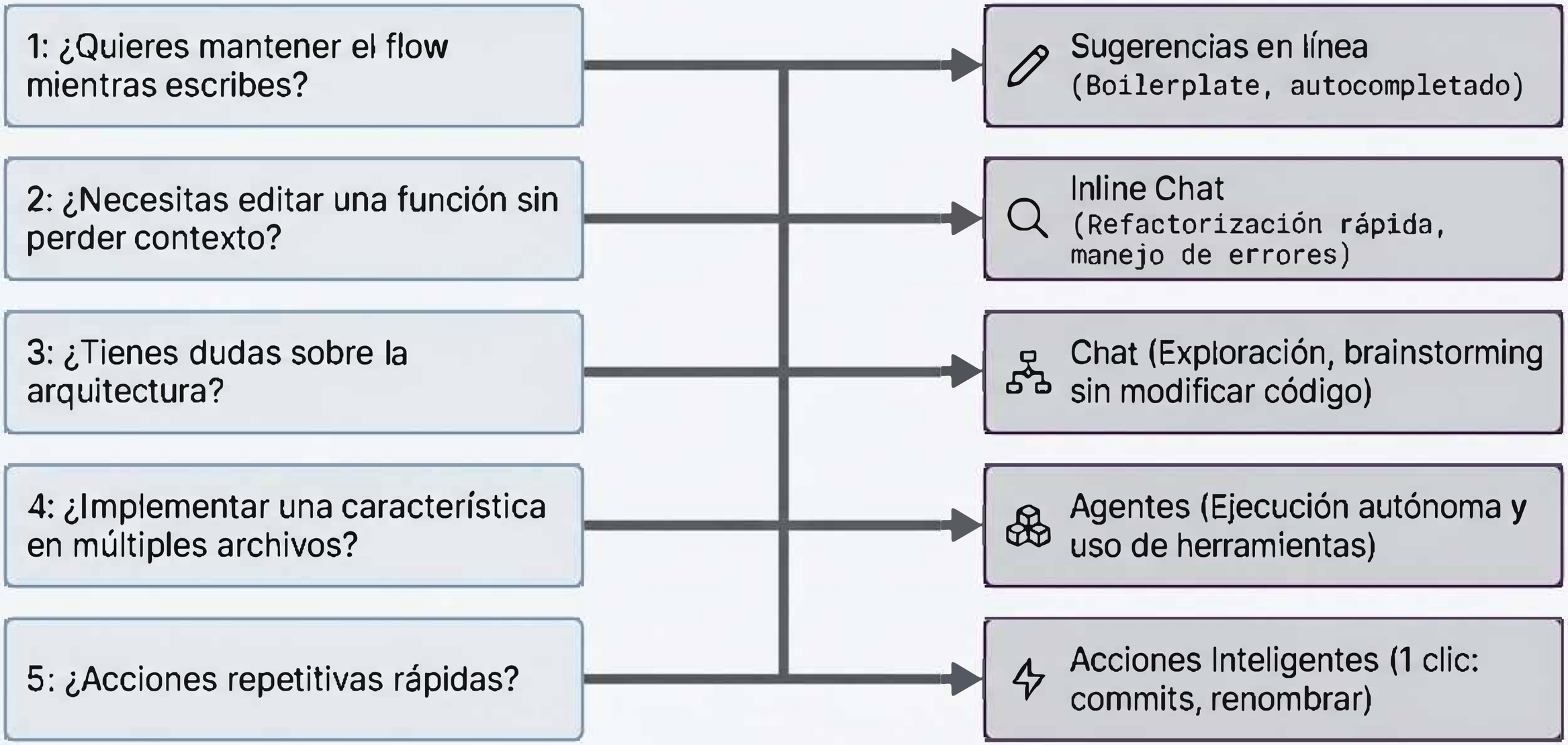
Subagentes en Paralelo



Delegación Eficiente: Investigación y análisis delegados a subagentes aislados que operan simultáneamente.

Reporte Estructurado: Reportan hallazgos al agente principal sin saturar el historial de la conversación principal con ruido o contexto irrelevante.

Herramientas adecuadas según el tipo de trabajo



Ingeniería de Prompts Efectiva

BLOQUE DEL PROMPT

Contexto Específico: Librerías, lenguajes, frameworks.

```
Stack: Python 3.10, FastAPI, SQLAlchemy.
Use AsyncIO. Avoid synchronous calls.
```

Restricciones Claras: Qué no hacer, convenciones a seguir.

```
NO usar `print()` para depurar.
Seguir PEP 8.
Manejar excepciones con `try-except`.
```

Salida Esperada: Casos de prueba o criterios de aceptación para validación.

```
Generar endpoint REST para crear usuario.
Incluir test unitario con pytest.
Validar JSON de entrada.
```

Reglas de Oro

1. Divide tareas complejas en pasos pequeños.
   Reduce el margen de error y facilita la depuración.
2. Evita la ambigüedad (Ej. Cambia "Mejora esto" por "Reduce la complejidad temporal").
   Instrucciones precisas generan resultados precisos.
3. Instruye al agente a hacer preguntas aclaratorias antes de codificar.
   Asegura la alineación y evita suposiciones incorrectas.

Higiene de Sesiones y Gestión de Contexto

✅ **Do's** (Qué Hacer)	❌ **Don'ts** (Qué Evitar)
✅ Utilizar menciones explícitas (`#file`, `@workspace`) para enfocar al modelo.	❌ No reciclar un mismo chat para explorar problemas de diferentes microservicios.
✅ Iniciar nuevas sesiones para tareas no relacionadas.	❌ No dejar que el historial crezca con respuestas previas fallidas o irrelevantes (Contaminación de contexto).
✅ Utilizar sesiones paralelas para ejecutar tareas independientes simultáneamente.	

💡 **Consejo:** Si las respuestas del modelo comienzan a degradarse, el límite de contexto está saturado. **¡Abre un nuevo chat!**

Flujo de Trabajo: Planificar Antes de Implementar

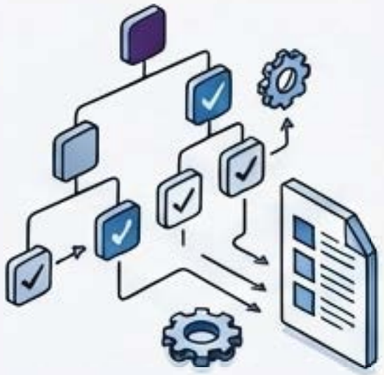
1. Explorar

Usar el agente en modo Ask o un subagente para leer código y entender la base actual.

```
> @workspace read code
```

2. Planificar

Usar el Plan Agent para generar una lista estructurada de tareas y decisiones de arquitectura.



```
> /plan generate-tasks
```

3. Implementar

El agente codificador ejecuta el plan. La lista de tareas se actualiza automáticamente.

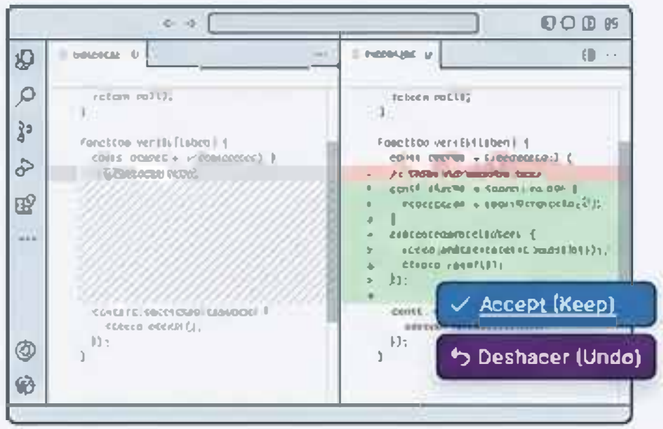
```
> executing_plan.py
  --update-status
```

4. Revisar

Validar cambios con pruebas y puntos de control (Checkpoints).

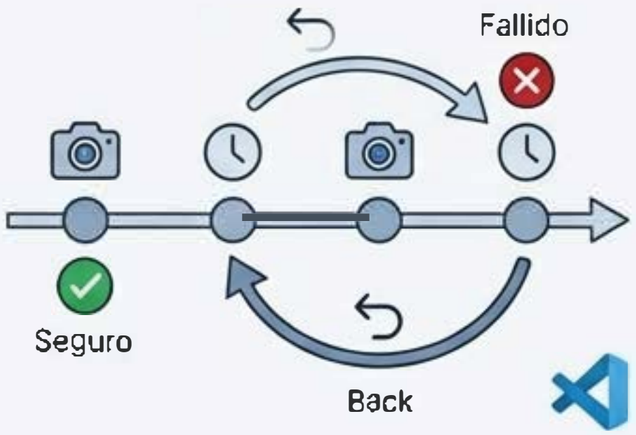
```
> run_tests.sh && git
  checkpoint
```

Revisión, Seguridad y Control



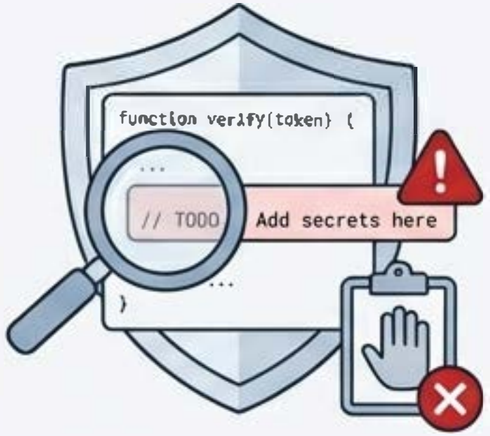
Diffs en Línea

Visualiza cambios propuestos directamente en el editor. Usa los controles superpuestos para Aceptar (Keep) o Deshacer (Undo) línea por línea.



Checkpoints

VS Code crea instantáneas automáticas. Si el agente toma un camino incorrecto, retrocede a un estado seguro en lugar de intentar arreglar errores en cascada.



Límites de Confianza

Revisa siempre el código generado buscando vulnerabilidades. Nunca pegues credenciales ni secretos en un prompt.

Escalando con Agentes Personalizados y MCP

Personas Especializadas

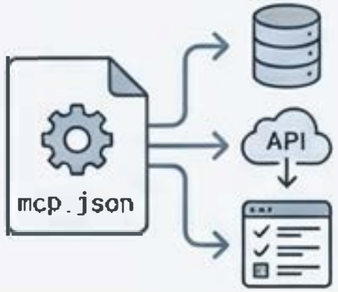
Crea agentes con roles específicos (ej. Auditor de Seguridad, Arquitecto). Tienen sus propias instrucciones y herramientas limitadas.

Agent Skills

Carpetas con instrucciones y scripts reutilizables que funcionan a través de VS Code y GitHub Copilot CLI.

Servidores MCP

Configura mcp.json para conectar agentes a sistemas externos (Bases de datos, APIs de la empresa, trackers de tickets).



Flujo Avanzado: Ingeniería de Contexto

1. Curar Contexto (Documentos Vivos)

Mantener la visión del producto y pautas en instrucciones personalizadas. Trátalas como código que evoluciona.

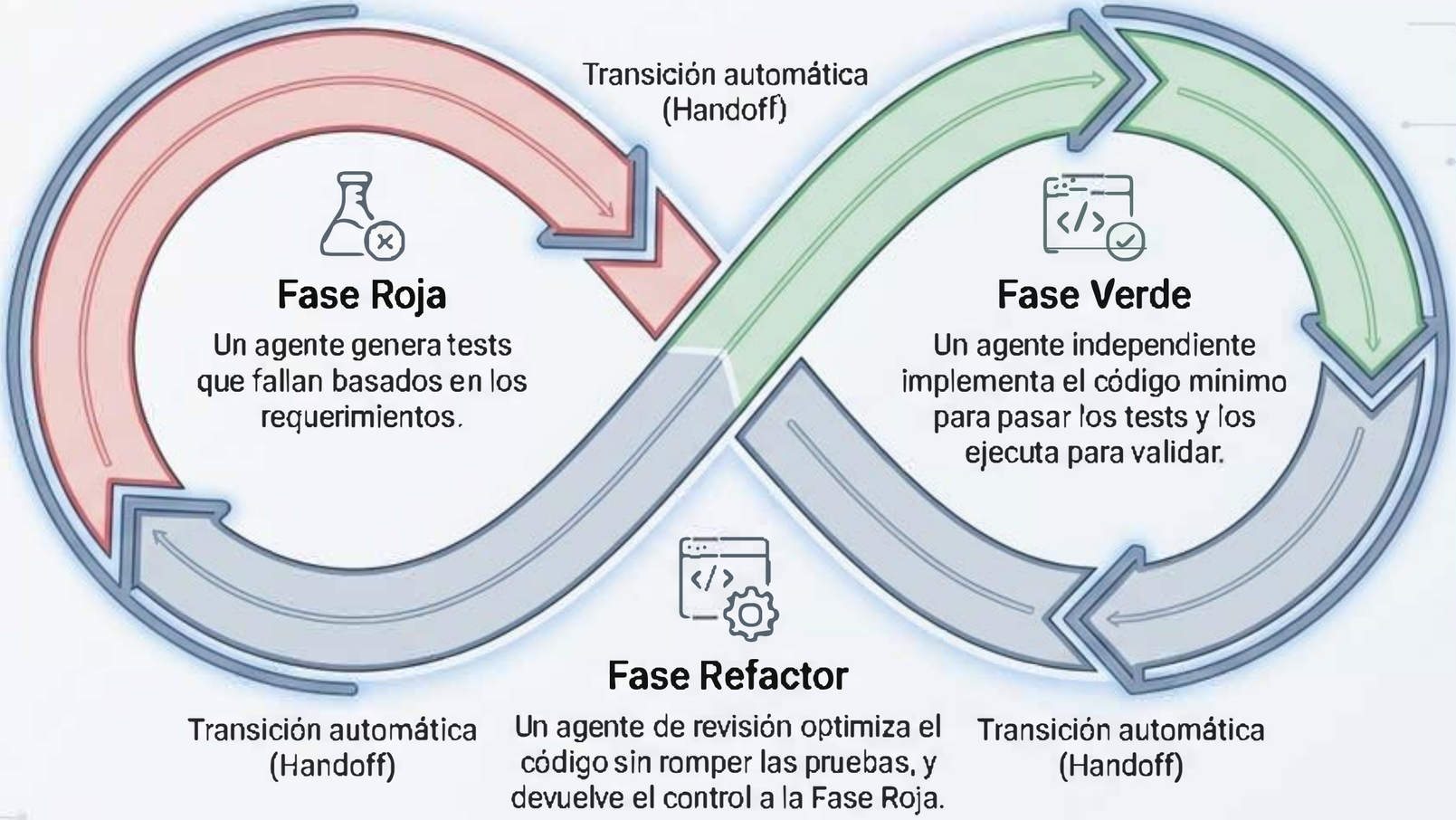
2. Generar Plan (Persona de Planificación)

Usar un agente personalizado estrictamente configurado para diseño de soluciones, evitando que el modelo salte directo a programar.

3. Generar Código (Límites Estrictos)

El agente de implementación genera el código basado únicamente en el plan aprobado, aplicando las reglas curadas.

Flujo de Trabajo: TDD Asistido por IA



Takeaway: El dominio total de Copilot ocurre cuando diseñas sistemas de agentes que colaboran y se auditan entre sí.